Arithmetic-Fair Comparisons of Exact FIR Software: A Cortex-M0 Model Case Study

Harsh Upadhyay
Reg No.: 240911650
Roll No.: 57

Aishwarya Singh
Reg No.: 240911232
Roll No.: 24

Aryan Devpura
Reg No.: 240911108
Roll No.: 12

***Abstract*—Exact FIR comparisons depend on the arithmetic-range information supplied to each implementation. We study this dependence using established direct and transposed realizations on a pinned Cortex-M0 simulator and CPU energy model. Replacing wide direct accumulation with two certified int32 partial sums overturns two of four original advantages of Hcub, a transposed implementation with shared constant products. A panel frozen before performance evaluation extends this finding to two fresh wide 48-tap arrays: stronger direct controls reverse Hcub's advantage in 24 of 32 complete conditions. Yet Hcub loses to ordinary transposition in all 32, separating product sharing from realization choice. Eight unavailable library cases leave the full three-array panel incomplete. Saved events expose the tradeoff between arithmetic savings and RAM traffic. A matched direct-kernel intervention makes a one-output bound visible inside the callee, removes identified preparation for another output, and lowers GCC15 modeled cost by 31.44%, reversing the compiler ranking for that pair. These findings support a bounded empirical case study of arithmetic controls, state representation and compiler knowledge, without a new FIR algorithm, general selection rule or physical-energy advantage.**

I. Introduction

An exact FIR comparison should give each implementation access to the same arithmetic-range information. On a processor with native 32-bit arithmetic, a full convolution sum may require 64 bits even when separately accumulated groups fit in 32 bits. Comparing a transposed implementation that exploits these bounds with a direct implementation that uses wide arithmetic throughout can therefore confound the realization with the arithmetic treatment.

Product sharing introduces a second distinction. Transposition exposes the products of one input sample with many constants, allowing common shift/add expressions to be reused. It also replaces input history with weighted suffix states, changing their widths and lifetimes. A library-relative advantage can therefore combine sharing, state representation and other implementation choices. Ordinary transposition, with the same state equations but unshared products, provides the control needed to isolate sharing.

We examine these distinctions on a pinned Cortex-M0 simulator using a configuration-specific CPU energy model. The original nonsymmetric 48-tap filter, `speech_minphase48`, supplies four GCC10/GCC15 and block1/block64 comparisons. Two certified native partial sums make direct accumulation less costly than Hcub under GCC10; Hcub remains less costly under GCC15. This compiler split motivates a matched intervention that exposes a one-output bound inside the direct callee and examines the resulting machine code.

A prospectively frozen panel tests whether the baseline effect extends to fresh coefficient arrays. It includes one native-sum and two widening-required 48-tap designs, with paired input amplitudes and measured stream regions. The original comparison motivated this panel; its four-cell reporting rule is retrospective. The new panel and intervention were fixed before their target costs were observed (Section 4).

The contribution is a controlled empirical account of three effects: arithmetic correction changes the direct/Hcub comparison; sharing loses to ordinary transposition on both fresh wide arrays; and fixed-block knowledge changes the direct compiler ranking. Exact event sensitivity and portable saved evidence support these findings. Range analysis, transposition, Hcub and specialization are established techniques; field priority and a general implementation rule remain unestablished.

II. Closest prior work

Range-aware DSP code generation is established. De Coster et al. [1] use magnitude information in compiled bit-true DSP simulation to avoid unnecessary overflow handling. Ménard and Sentieys [6] select software word lengths and instructions under a signal-to-quantization-noise ratio constraint, accounting for arithmetic and transfers. Our full-int16 exact-output contract differs from their accuracy constraint; it does not make width selection a new technique.

FIR generation and energy-oriented implementation selection also precede this study. Gačić et al. [2] generate FIR software with SPIRAL and show that blocking and computation order change runtime at equal arithmetic count. D'Alberto et al. [3] optimize fixed-point XScale transforms for runtime and physically measured energy. These results motivate examining compiled behavior rather than arithmetic count alone. Their hardware measurements do not validate our Cortex-M0 model estimates.

Hcub provides established product-sharing graphs, rather than an executable energy objective. Voronenko and Püschel [4] discuss software concerns beyond addition count and explicitly leave register-pressure optimization outside their study. We compare Hcub with ordinary transposition to test its incremental benefit after compilation, separately from the effect of changing realization.

The cost analysis and specialization use established tools. Nikov et al. [5] supply the configuration-specific event weights; our affine reweighting is standard sensitivity analysis. GCC documents constant propagation, cloning and interprocedural optimization [7], but those documents cannot identify the pass responsible for our historical binaries. Saved-evidence portability follows established artifact practice [10]. The contribution is the controlled empirical finding, not these techniques. This targeted comparison is not an exhaustive priority search.

III. Exact contract and controlled representations

A. Numeric semantics

All filters have 48 frozen signed-int16 coefficients, ordered so that $h[0]$ multiplies the newest sample. Every candidate must accept all $x \in [-32768, 32767]$, including -32768, regardless of the amplitude used for performance measurement. Initial history is zero. For requested samples only,

$$\begin{aligned} S[n] &= \sum_{k=0}^{47} h[k]x[n-k], \\ Q(s) &= \min\{32767, \max\{-32768, \lfloor s/32768 \rfloor\}\}, \\ y[n] &= Q(S[n]), \qquad x[n] = 0 \text{ for } n < 0. \end{aligned}$$

Exactly one final quantization and saturation is permitted. There is no intermediate truncation, saturation or appended convolution tail. In particular, $Q(-1) = -1$; truncation toward zero at that boundary changes the contract. The generated quantizer clamps before narrowing and implements negative floor using C division without assuming arithmetic signed right shift.

For each coefficient, exact independent-input product endpoints are $l_k = \min(-32768h_k, 32767h_k)$ and $u_k = \max(-32768h_k, 32767h_k)$. Summing those endpoints gives tight intervals for the full sum and any specified group. A sign-aligned legal sample window attains each endpoint. A sufficient group-prefix certificate also uses $32768 \sum_{k \in G} |h_k| \le$ INT32_MAX: every product and accumulated prefix within G fits int32. Full intervals, prefix bounds, transposed suffix bounds and Hcub primitive/restored-product identities are separate certificates. C evaluation widths must respect them; widening after an overflowing addition is too late.

B. The original arithmetic correction

The correction uses native partial sums without changing the direct history layout or coefficient order. The coefficient L1 norm is the sum of absolute coefficient values. Fixed groups $[0, 2)$ and $[2, 48)$ have coefficient L1 sums 12,778 and 59,731, hence absolute prefix bounds 418,709,504 and 1,957,265,408. Both fit int32. The full L1 sum is 72,509, with absolute bound 2,375,974,912; legal sign-aligned windows attain 2,375,922,255 and −2,375,955,060, establishing that full-sum int32 overflow is possible. Each completed partial is converted separately before the int64 reduction:

```
int64_t exact_sum = (int64_t)partial0 + (int64_t)partial1;
y[j] = quant(exact_sum);
```

This split was fixed for the arithmetic-control experiment. It establishes legality, without claiming an optimal partition or direct implementation.

Ordinary transposition computes the current sum from $h[0]x + z_0^{old}$ and updates z_i from $h[i+1]x + z_{i+1}^{old}$, ending with the last product. Ascending updates preserve the required old states. Suffix bounds justify each state's width. Hcub adds independently checked shared expressions and demand-driven emission near their consuming stage. Safe exact operations replace problematic signed shifts.

The controls isolate different effects (Figure 1). Partial versus wide direct changes the accumulation width while retaining input history. Hcub versus ordinary transposition changes product sharing while retaining suffix equations and layout. Hcub versus direct also changes representation, and comparison with CMSIS changes further implementation choices. Correctness checks reconstruct the intended history or suffix values rather than compare unlike state bytes.

IV. Measurement, chronology and prospective questions

A. Target and event models

The target is Cortex-M0 Thumb at `-O2`, separate compilation and no LTO. GCC10 reports 10.3.1 20210621; GCC15 is Arm GNU Toolchain 15.2.Rel1, reporting GCC 15.2.1 20251203. The simulator is pinned to commit `c55ea1b192 6b792d7aabb7e3495b55a7b69ef134`. Code and coefficients reside in Flash, with state/input/output in SRAM. Measurement includes dispatch, recurring calls, copies, convolution, quantization, output stores and retained history/state movement, including a final short call. Initialization, checking and printing lie outside. The audited decoder removes the closing marker instruction consistently.

Let N_I include all instructions, N_M native multiplication instructions, B taken branches, R and W SRAM reads/writes, F Flash data reads and N_{BL} branch-with-link instructions. Let $N_{POP,PC}$ count returns that pop PC from the stack. The zero-wait, single-cycle-MUL cycle estimate, using Arm's instruction timings [11], is

$$T = N_I + R + W + F + 2B + N_{BL} + N_{POP,PC}.$$

Arm assigns a return-by-POP $4 + N$ cycles, including the PC transfer in N. Instruction, read and taken-branch counts supply $3 + N$; the final term supplies the remaining cycle. The corrected timing report rederives this count from saved logs and screens the linked scalar instruction subset. It supersedes the historical inventories' cycle fields, which omitted that term. Energy weights and measured events are unchanged; the first paper and archive retain their historical values.

Nikov et al.'s [5] 20 MHz, prefetch-off, zero-wait weights give modeled CPU energy in nJ:

$$\begin{aligned} E = {} & 0.964258(N_I - N_M) + 1.652455N_M \\ & + 2.091986B + 1.109833R + 0.650563W + 0.633621F. \end{aligned}$$

TABLE I
THE FIVE PRINCIPAL ORIGINAL FORMS; ALL SATISFY THE SAME FINAL-OUTPUT CONTRACT.

Short name	Original artifact identifier	Controlled role
CMSIS	`cmsis64_unroll`	Historical unrolled wide library control
Wide direct	`spec_direct`	Specialized chronological direct, wide sum
Partial direct	`partial_direct32x2`	Same direct history, two native partials
Ordinary transposed	`trans_const`	Bounded suffix states, ordinary products
Hcub	`trans_hcub_demand`	Same suffix equations, shared constant products

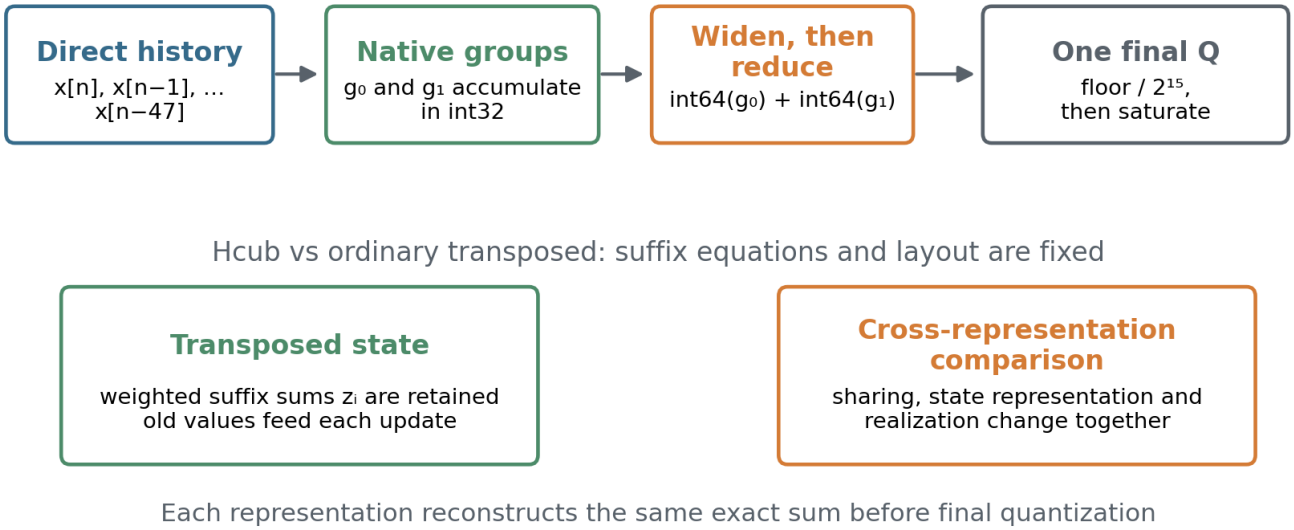


Fig. 1. Exact representations and controlled contrasts. Each direct native group widens before reduction; both representations keep the exact sum until one final quantizer. Direct history and weighted suffix states have different meanings and lifetimes. The cross-representation comparison combines sharing and realization.

The reported 2.80% mean absolute percentage error (MAPE) describes aggregate validation of that model, not uncertainty in our pairwise margins. Fitted weights can absorb correlations and residual effects; they are not separately measured physical event prices. We preserve integer events and exact decimal products for reproducibility. The 0.000001 nJ tolerance used to read historical floating-point data concerns serialization only.

B. Correctness checks and study chronology

The arithmetic correction is a retrospective control. The original protocol covered three designs, two input guarantees, two blocks and two compilers. Observed unequal width treatment motivated the fixed two-group correction and then the compiler investigation. An earlier source-level state-traffic estimate was withdrawn because register retention and grouped transfers could reduce it; no candidate had been excluded using that estimate. Previously inspected filters remain development evidence. The fresh panel and intervention were frozen before their target costs were observed.

The original comparison uses a saved 257-sample `adc_raw` stream with zero initial history. Block1 makes 257 calls; block64 makes four full calls and a final one-sample call. Let $r = 100(E_H/E_P - 1)$, where H is Hcub and P is partial direct. The retrospective all-four superiority claim, H_R, requires $r < 0$ in every compiler/block cell. A tie defeats strict superiority; missing or invalid evidence leaves the assessment unavailable. This deterministic rule does not involve significance testing.

Correctness admission combines checks that address different failure modes. Arithmetic certificates establish full-domain source legality. Finite UBSan checks and target output/state/guard checks test implementation behavior. The original independent check covered 1,028 new, 6,168 historical and 722 signed-edge output comparisons. Emitted-code inspection confirmed native partials and a wide final reduction in all four corrected binaries. Hashes bind measured events to checked sources and binaries. Library paths additionally require pinned source/toolchain semantics; output agreement alone is insufficient. The CMSIS packed helpers rely on GCC's sign-extending signed right shift, modulo conversion to signed widths and documented signed-left-shift extension. These compiler dependencies are explicit in both pinned GCC versions [12].

C. Saved-event analysis

We use saved events to identify the work saved or added by each comparison. For each compiler/block, the three oriented pairs are partial minus wide direct, Hcub minus ordinary transposition and Hcub minus partial direct. All use the same 257 outputs. With $v = (N_I - N_M, N_M, B, R, W, F)$ and the weights above, $\Delta E = w \cdot \Delta v$; negative favors the left form. Six-event dominance requires all coordinates nonpositive with at least one strictly negative; mixed signs preclude it. Resources are excluded.

A predeclared sensitivity check scales both RAM weights by λ while holding the other four weights and all events fixed. For $\lambda \geq 0$,

$$\Delta E(\lambda) = A + B_{RAM}\lambda,$$
$$B_{RAM} = 1.109833\Delta R + 0.650563\Delta W.$$

We examine the arbitrary interval $[0.5, 2]$ to locate algebraic ranking boundaries. It is not calibrated to physical prices, model uncertainty or another hardware configuration. Exact decimal products give rational boundaries $-A/B_{RAM}$; displayed roots are rounded. Linearity makes endpoint signs sufficient to decide strict ordering over the closed interval, with any tie defeating strictness. The artifact retains all 12 vectors, components, resource deltas and root classifications.

D. Frozen synthetic panel

The fresh panel contains three synthetic 48-tap arrays chosen to span distinct arithmetic regimes: one full sum fits int32 and two require widening. With NumPy 1.26.4/SciPy 1.15.3, we use `firwin(95, cutoff, fs=16000, window=window, scale=True)`, homomorphic `minimum_phase(half=True,n_fft=32768)`, target-L1 scaling and `np.rint`. The half-length conversion changes the prototype magnitude response [8]; these arrays are arithmetic controls, not representative application filters.

Two rules determine the wide designs' direct partitions. The first cut is the smallest legal two-group boundary; the balanced cut minimizes the larger group L1, breaking ties by the smaller boundary. Legal sign-aligned windows attain the full-sum endpoints, proving possible int32 overflow in both wide arrays. Coefficients, certificates, graphs, inputs and the case list were frozen by hash before performance evaluation.

Each array has six implementations. All include wide direct, first certified direct, ordinary transposed, Hcub and standard unrolled CMSIS. The sixth is fast CMSIS for the native array or balanced direct for a wide array. Candidates receive identical coefficients, full-int16 semantics and block knowledge. Generation uses fixed recipes: two direct partition rules for each wide array and one Hcub graph/emission schedule. Candidates are not replaced based on observed target cost; equal optimization effort or exhaustive search is not claimed. Pinned CMSIS-DSP v1.14.4 paths use reversed coefficients and required storage, with separate source/compiler eligibility and no fast-path input scaling [9]. Each Hcub graph comes from one fixed `acm -b 15 -seed 1 -nofs -gc` run on sorted odd fundamentals.

Two paired inputs test amplitude, and two measured regions test stream conditions. A frozen 1089-sample full-range linear congruential generator (LCG) stream supplies the master input; samplewise `full//32` supplies its ADC partner. Startup measures the first 257 outputs from zero history. Post-startup processes 64 samples outside the measured region, retains history, then measures offsets [64,1089): 1025 outputs. Both block sizes align at that boundary; block64 ends with one short call in each region. Checks include warmup. Because the regions differ in segment and length, this contrast does not isolate startup cost or establish steady state.

A condition fixes the array, compiler, block size, input amplitude and region; a performance identity additionally fixes the implementation. The frozen inventory has 288 identities: 3 arrays × 6 forms × 2 compilers × 2 blocks × 2 amplitudes × 2 regions. Separate correctness admission uses at most 72 target identities on a frozen 194-sample zero/extremal/edge stream, plus host UBSan checks with mixed block partitions. Images and conservative live call chains must fit 64 KiB Flash/8 KiB RAM. Unavailable and failed cases remain explicit; conditions are not independent statistical samples.

The primary question is whether a stronger direct comparator reverses an old-control Hcub advantage. For a fixed-compiler condition, let D_{old} be the minimum eligible non-transposed old control: wide direct, standard CMSIS and fast CMSIS where eligible. Let D_{new} add the available certified direct forms. A reversal satisfies $E_H < D_{old}$ but $E_H \geq D_{new}$. The prospective extension requires at least one such complete condition in either wide array. One admitted witness supports this existential extension. Zero witnesses refute it only if every required wide condition is complete; otherwise it remains unresolved. Missing native cases separately leave the larger three-array panel incomplete. All ties retain every minimizer.

Additional comparisons distinguish implementation choices from that reversal predicate. We report all-form winners, certified/wide and Hcub/ordinary percentages, events, resources and paired input/region contrasts. A separate compiler-selectable analysis minimizes each form across compilers before comparing forms. Exact reference sums count the clamp conditions $S \geq 1073741824$ and $S \leq -1073741824$ in each measured region; output endpoints alone cannot identify those conditions.

E. Matched block-knowledge intervention

The intervention tests whether callee-visible block knowledge changes the original direct compiler comparison. It uses the original block1 fixture and 257-sample stream, with each compiler building a general and a specialized kernel. Specialization replaces only the three expression uses of runtime `b` inside `probe_process` with literal `1` and adds `(void)b;`. Every call satisfies $b = 1$. Arithmetic, groups, quantizer, history algorithm, wrapper, ABI, flags and separate compilation without LTO remain fixed.

TABLE II
FROZEN ARITHMETIC REGIMES, INDEPENDENTLY PREPARED BEFORE TARGET COSTS. CUTS GIVE FIRST-LEGAL / RANGE-BALANCED TWO-GROUP BOUNDARIES.

Design	Cutoff/window	Target / actual L1	Exact full-sum interval	Direct cuts
`native_lp1800`	1800 Hz / Hamming	50000 / 50001	[−1638421124,1638394411]	Whole native sum
`wide_lp2800`	2800 Hz / Hamming	100000 / 100002	[−3276839180,3276791890]	4 / 5
`wide_lp4600`	4600 Hz / Blackman	110000 / 110003	[−3604545833,3604500772]	3 / 4

Historical comparability requires the general controls to reproduce the saved loadable binaries and events exactly. Four signed-edge target checks and repeated-one-sample host checks support correctness. Define the signed compiler gaps $G_g = E_{15,general} - E_{10,general}$ and $G_s = E_{15,specialized} - E_{10,specialized}$; the historical $G_g = 70469.699925$ nJ. The predeclared success criterion requires three observations: identified preparation for another output disappears from reviewed code, GCC15 modeled cost falls, and $G_s < G_g$. Event, cycle and resource deltas document the full change. Attributing that total to a particular transfer subset would require further accounting.

V. ORIGINAL COMPARISON AND EVENT ANALYSIS

A. Width correction changes two decisions

Native partials lower modeled CPU energy by 23.44% to 41.98% relative to wide direct across the four original cells. Every one of the six event coordinates is no worse, with strict improvements in each cell: partial direct is six-event-dominant in these saved comparisons. This statement concerns events, not physical energy or resource dominance. The tables retain all five implementations and the corrected ranking.

TABLE III
ORIGINAL MODELED CPU ENERGY, NJ/REQUESTED OUTPUT; ALL PROCESS THE SAME 257 SAMPLES. DISPLAY ROUNDING DOES NOT DETERMINE WINNERS.

Variant	GCC10 b1	GCC10 b64	GCC15 b1	GCC15 b64
CMSIS	1537.21	905.94	1503.18	838.85
Wide direct	1543.04	990.40	1712.07	1023.10
Partial direct	**1036.55**	**575.81**	1310.75	593.59
Ordinary transposed	1045.78	605.66	1039.99	602.87
Hcub	1081.44	604.62	**995.43**	**562.55**

TABLE IV
$100(E_H/E_{named} - 1)$. BOTH HISTORICAL COMPARISONS FAVOR HCUB IN ALL FOUR CELLS. THE CORRECTED COMPARISON REJECTS H_R WITH TWO SURVIVING CELLS.

Compiler/block	Hcub / wide direct	Hcub / CMSIS	Hcub / partial direct
GCC10/1	−29.91%	−29.65%	+4.330785%
GCC10/64	−38.95%	−33.26%	+5.002907%
GCC15/1	−41.86%	−33.78%	−24.056759%
GCC15/64	−45.01%	−32.94%	−5.229188%

For GCC10/block64, partial direct executes 129,481 instructions and 2,827 MULs; Hcub executes only 116,139 instructions and 257 MULs. Yet Hcub incurs 10,994 additional RAM reads and 18,235 additional writes. Its complete modeled cost is 604.62 versus 575.81 nJ/output. Graph nodes therefore do not determine the implementation winner.

Fixed versus selectable compiler decisions also differ. With GCC15 fixed, Hcub is 24.06% and 5.23% below partial direct. If either compiler may be chosen separately for each form, GCC15 supplies the best tested Hcub and GCC10 the best partial direct; the savings become 3.97% and 2.30%. These minima cover only the tested builds and are not calibrated physical-energy margins.

B. Sharing saves arithmetic but adds RAM traffic

Within the same transposed equations, Hcub costs more at GCC10/block1 and less in the other three nominal cells. Every incremental-sharing event vector is mixed: fewer native MULs and Flash reads coexist with additional RAM transfers. GCC10/block1 has no non-MUL instruction reduction at all.

The comparator changes the apparent size of the benefit. At GCC15/block64, Hcub's 32.94% saving against CMSIS falls to 6.69% against ordinary transposition and 5.23% against corrected direct. Only the ordinary-transposition comparison isolates sharing.

Memory diagnostics expose a separate tradeoff. Hcub and ordinary transposition allocate equal state, but Hcub's process frame is 72 bytes larger in every cell. Its selected code/coefficient symbols are smaller. Partial and wide direct also allocate equal state; partial direct has smaller frames and selected symbols in every cell. Appendix A retains all values. State allocation, process frames and selected symbols measure different things and must not be summed as memory use.

C. Sensitivity to RAM weights

RAM reweighting separates a consistent arithmetic benefit from conditional sharing benefits. Partial minus wide direct has $A < 0$ and $B_{RAM} < 0$ in every cell, so it remains cheaper for all $\lambda \geq 0$. All eight Hcub comparisons have $A < 0$ and $B_{RAM} > 0$: extra RAM cost eventually outweighs the non-RAM benefit. The exact analysis reconciles all 12 nominal differences.

Only GCC15/block1 keeps Hcub below both stronger controls throughout [0.5,2] (Figure 2). GCC10/block64's sharing benefit changes sign near 1.019679; GCC15/block64 crosses against both controls before 2. These are algebraic boundaries for fixed events, not physical robustness bounds.

TABLE V

HCUB MINUS ORDINARY TRANSPOSITION, WHOLE 257-OUTPUT STREAM; PERCENTAGE DENOMINATOR IS ORDINARY TRANSPOSITION. VECTOR COORDINATES ARE $(\Delta nonMUL, \Delta MUL, \Delta B, \Delta R, \Delta W, \Delta F)$; NONMUL IS $N_I - N_M$.

Compiler/block	Δv	ΔE, nJ	Relative
GCC10/1	(0, −2570, 0, 9509, 5397, −1028)	+9166.318770	+3.410536%
GCC10/64	(−9324, −2570, 0, 9257, 5145, −1028)	−268.042614	−0.172202%
GCC15/1	(−17990, −2570, −257, 7196, 5140, −1028)	−11452.561472	−4.284890%
GCC15/64	(−17234, −2570, −5, 7196, 4888, −1028)	−10360.343828	−6.686823%

TABLE VI

EXACT COEFFICIENTS, ROUNDED ROOTS; CELL ABBREVIATES COMPILER/BLOCK. THE FOUR NEGATIVE ROOTS LIE OUTSIDE THE PERMITTED $\lambda \geq 0$ DOMAIN. EXACT RATIONAL ROOTS AND ENDPOINT WINNERS ARE IN THE MACHINE-READABLE ANALYSIS.

Pair (left − right)	Cell	A, nJ	B_{RAM}, nJ	Root (display)	Strict lower on [0.5,2]?
Partial − wide	10/1	−87520.461808	−42645.678958	−2.052270	Yes
Partial − wide	10/64	−64652.689002	−41894.635425	−1.543221	Yes
Partial − wide	15/1	−62253.689535	−40885.154171	−1.522648	Yes
Partial − wide	15/64	−69714.748911	−40667.649816	−1.714256	Yes
Hcub − ordinary	10/1	−4898.171738	14064.490508	0.348265	No; higher throughout
Hcub − ordinary	10/64	−13888.913330	13620.870716	1.019679	No; changes sign
Hcub − ordinary	15/1	−22782.813560	11330.252088	2.010795	Yes
Hcub − ordinary	15/64	−21526.654040	11166.310212	1.927822	No; changes sign
Hcub − partial	10/1	−32436.319000	43973.275423	0.737637	No; changes sign
Hcub − partial	10/64	−16661.004146	24064.520307	0.692347	No; changes sign
Hcub − partial	15/1	−94335.682384	13297.172804	7.094417	Yes
Hcub − partial	15/64	−16351.634074	8374.315001	1.952594	No; changes sign

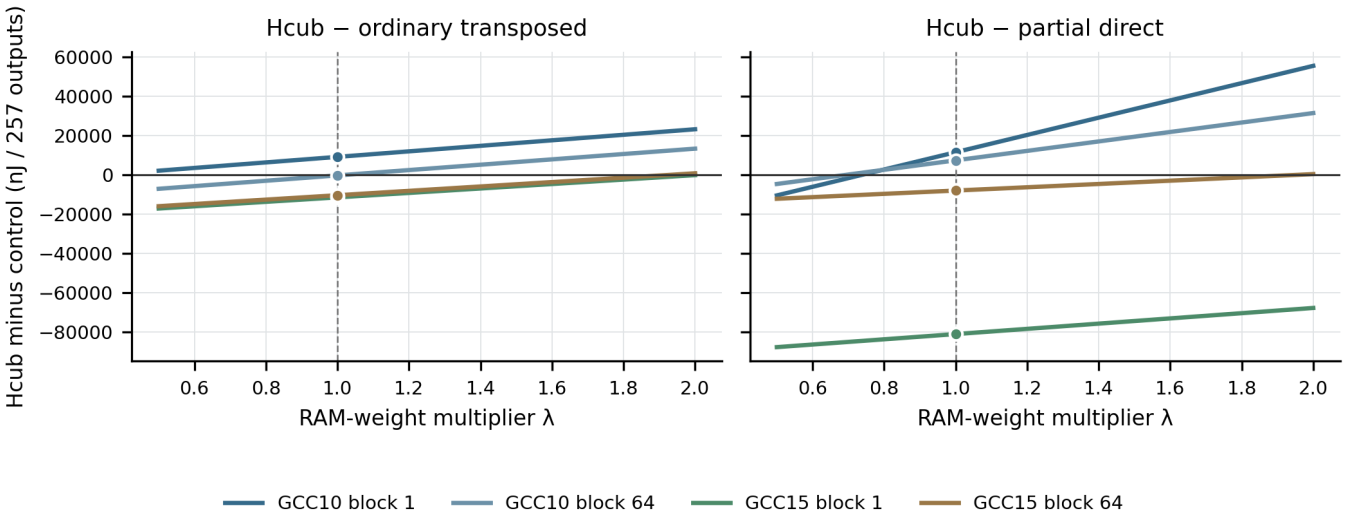


Fig. 2. Saved-event RAM-weight sensitivity for Hcub minus ordinary transposition and Hcub minus corrected direct. Negative differences favor Hcub. Lines reuse fixed 257-output event vectors; lambda scales both RAM weights on one arbitrary, uncalibrated path. Zero crossings are algebraic ranking boundaries, not physical uncertainty bounds.

VI. Compiler account and intervention

A. Explaining the original block1 binaries

The original block1 compiler difference is largely a stack-traffic difference. GCC15 executes 242,490 instructions, 43,182 RAM reads and 36,500 RAM writes; GCC10 executes 198,670, 29,304 and 17,225. Both use two native partials and a wide final reduction, eleven native MULs/output, nine saved/restored ABI words/call, and four stores/reloads at the partial-to-wide interface. Extra multiplications or wide per-tap arithmetic therefore do not explain the gap.

The binaries prepare differently for another output. GCC15 preloads history and maintains a sliding sample window in registers and stack slots, rotating samples before testing the output-loop bound. GCC10 loads mostly near use and carries one sample forward. For example, GCC15's `[sp,#40]` holds history sample 45, supplies coefficient 12990, then receives sample 46 for a possible next iteration. The callee remains general even though every call requests one output. Stack residence alone does not prove a register-allocation spill.

Checked paths reconcile the additional RAM traffic exactly. All 257 calls have $b = 1$; the copy-in and convolution loops iterate once. The 47-step history copy and alternative quantizer paths have no direct-SP access, and the latter rejoin before the common tail. SP is fixed, there are no internal calls, and GCC10's zero-block store is unreachable. GCC10 executes 9 loads/9 stores relative to SP per call; GCC15 executes 63/84. The difference is 13,878 reads and 19,275 writes. This path-based count supplements aggregate logs, which are not per-PC traces.

TABLE VII
Accepted additive account of the two original direct binaries.

GCC15 − GCC10, Total block1		Extra direct-SP transfers	Residual
Instructions	43820	33153	10667
RAM reads	13878	13878	0
RAM writes	19275	19275	0
Taken branches	131	0	131
Estimated cycles	77235	66306	10929
Modeled energy, nJ	70469.699925	59909.909673	10559.790252

Each extra transfer contributes an instruction and a RAM event. Together they account for 85.02% of the modeled compiler increase; the remaining instruction and branch cost stays explicit. MUL, Flash, Flash-write and BL deltas are zero. Checks bind the two rows to 29 evidence hashes, with manual value/path inspection supporting the account. This additive decomposition does not predict the saving from removing transfers or identify a compiler pass.

B. Fixed-block specialization reverses the compiler ranking

Specialization lowers modeled cost by 8.86% for GCC10 and 31.44% for GCC15. Both general controls reproduce the historical loadable binaries and all ten counter fields exactly. All four performance and four separate 361-sample signed-edge cases pass output/history/guard and raw-evidence checks.

The signed compiler gap changes from about +70,470 to −11,857 nJ. GCC15 becomes the less costly specialized direct build, satisfying both numerical criteria: its own cost falls and $G_s < G_g$.

Emitted-code inspection also confirms removal of the identified preparation. GCC15's clustered preloads, stack-resident sliding window and rotation tail disappear. It loads history near use and proceeds from the output store to the required 47-word history shift. GCC10 loses its carried-sample recurrence and output-loop control, while retaining the sample needed by the first history shift. Required retained-history movement remains in both builds.

The full saving belongs to the block-knowledge intervention. Copy-in and output-loop control, high-register saves/restores, stack locals and stack-materialized accumulator pairs also disappear; arithmetic scheduling and quantizer branches change. These accompanying effects prevent exclusive attribution to rotation removal. The earlier 85.02% additive account does not partition the intervention saving, and frame shrinkage alone is not traffic evidence. These pairs establish no ranking among other forms or physical saving.

TABLE VIII

THE FOUR FIXED INTERVENTION ROWS, 257 MEASURED OUTPUTS EACH. CYCLES ARE THE EVENT-DERIVED ESTIMATE. PROCESS-SYMBOL SIZES INCLUDE LITERAL POOLS; PROCESS FRAMES, CONSERVATIVE LIVE CALL-CHAIN BYTES AND LINKED FLASH-IMAGE BYTES ARE DISTINCT RESOURCE MEASURES. STATE ALLOCATION STAYS 98 BYTES, STATIC RAM ENDS AT 0x20000488, AND INITIAL SP IS 0x20001FF0 IN EVERY ROW. THE SEPARATE EDGE WORKLOAD SUPPLIES CORRECTNESS EVIDENCE ONLY.

Compiler/form	Total energy, nJ	Energy/output, nJ	Cycles/output	Symbol, B	Frame, B	Stack chain, B	Flash image, B
GCC10 general	266394.093850	1036.552894	1066.112840	1036	72	144	2166
GCC10 specialized	242800.957748	944.750808	969.112840	888	20	92	2018
GCC15 general	336863.793775	1310.754061	1366.638132	1372	240	312	2490
GCC15 specialized	230943.514627	898.612897	921.108949	884	16	88	2002

TABLE IX

COMPLETE COST-COORDINATE AND CYCLE REDUCTIONS OVER THE 257-OUTPUT STREAM. VECTOR COORDINATES ARE $(\Delta nonMUL, \Delta MUL, \Delta B, \Delta R, \Delta W, \Delta F)$. INSTRUCTION DELTAS EQUAL NONMUL DELTAS; FLASH-FETCH DELTAS ARE 17,219 AND 72,605. BL AND FLASH-WRITE DELTAS ARE ZERO. DIAGNOSTIC RAW TICK DELTAS FALL BY 23,901 AND 112,828, RESPECTIVELY; THEY ARE NOT THE CYCLE ESTIMATE.

General − specialized	Δv	Estimated cycles	Modeled energy, nJ
GCC10	(17219, 0, 257, 3855, 3341, 0)	24929	23593.136102
GCC15	(72605, 0, 645, 17733, 22873, 0)	114501	105920.279148

VII. RESULTS ON THE FRESH SYNTHETIC PANEL

A. Stronger direct controls reverse Hcub comparisons

Both fresh wide arrays reproduce the baseline effect. Strengthening direct arithmetic reverses Hcub's old-control advantage in 24 of 32 complete fixed-compiler conditions: all 16 GCC10 conditions and eight GCC15/block64 conditions. The eight GCC15/block1 conditions retain its advantage over the stronger nontransposed set. Thus the predeclared existential extension has fresh witnesses under both amplitudes and both measured regions.

The full three-array panel remains incomplete. Eight native fast-CMSIS/block64 performance identities are unavailable because pointer-object aliasing is unresolved; they count as neither wins nor losses. The inventory admits 280 distinct performance binaries with zero failed identities. Separate correctness checks admit 70 of 72 identities, with that same form unavailable under each compiler. Missing cases were not replaced by different arrays, implementations or flags.

For Wide2800/GCC10/block64/ADC/post-startup, Hcub costs 661.31 nJ/output, versus 904.01 for D_{old} and 515.47 for D_{new}. The stronger direct control changes the comparison. Ordinary transposition can still win a condition with such a reversal: the predicate concerns the nontransposed controls, not the all-form minimum. Figure 3 shows both margins.

Across 40 complete fixed-compiler conditions, ordinary transposition wins 16, first direct alone eight, balanced direct alone four, Hcub four, and first/balanced direct tie in eight. These disjoint counts include ties once.

Allowing each form to choose its compiler changes the decision. Of 24 compiler-selectable conditions, 20 have known minima; four native/block64 conditions remain unknown. Reversals occur in eight of 16 complete wide conditions, all at block64. Ordinary transposition wins all eight wide/block1 conditions; first/balanced direct tie in all eight wide/block64 conditions. Hcub wins the four complete native/block1 conditions.

B. Sharing loses on both fresh wide arrays

Every admitted certified direct comparison improves on its wide counterpart, including the native-fit design. The first/balanced partition choice can change which direct implementation wins: it ties under GCC10 for Wide2800 and under GCC10 for both wide block64 groups, favors first under GCC15 for Wide2800 and favors balanced under GCC15 for Wide4600. No claim of optimal partitioning follows.

Hcub costs 0.12% to 7.66% more than ordinary transposition in **all 32** fresh wide conditions. Its original GCC15 sharing benefit therefore does not extend to either new wide array. The native array retains the compiler split: Hcub costs more under GCC10 and less under GCC15. Exact events and per-condition costs remain in the panel results and summary.

C. Workload boundaries and admitted resources

Block64 lowers cost/output in all 136 known within-form block pairs, by 31.554538% to 55.482311%; eight pairs are unknown. Full-range input is equal in cost to its paired ADC input in 28 of 140 known amplitude pairs and lower in 112, with changes from −0.046876% to zero; four are unknown. Post-startup cost/output is lower in 126 of 140 known regime pairs and higher in 14, spanning −0.375190% to +0.025079%; four are unknown. These exact deterministic pairs are workload contrasts, not independent statistical samples. Neither amplitude nor measured regime changes the known minima or reversal statuses in the grouped table. The small cost changes still deserve reporting rather than an assumed monotonic startup benefit.

The source quantizer reaches neither clamp threshold on ADC inputs or the native full-range input. Wide2800/full reaches the positive/negative thresholds 6/6 times in startup

TABLE X

EVERY FIXED DESIGN/COMPILER/BLOCK GROUP, EACH COVERING ADC/FULL × STARTUP/POST-STARTUP. MINIMA AND REVERSAL STATUS ARE INVARIANT ACROSS THOSE FOUR CONDITIONS. THE REVERSAL USES D_{old} AND D_{new}, NOT THE ALL-FORM MINIMUM; ORDINARY TRANSPOSITION IS EXCLUDED FROM BOTH NONTRANSPOSED SETS. NATIVE, WIDE2800 AND WIDE4600 ABBREVIATE THE FROZEN DESIGN IDENTIFIERS.

Design	Compiler/block	Reversals / four cells	All-form minimum
Native	GCC10/1	Not a wide-design test	First direct
Native	GCC10/64	Unknown	Unknown
Native	GCC15/1	Not a wide-design test	Hcub
Native	GCC15/64	Unknown	Unknown
Wide2800	GCC10/1	4/4	Ordinary transposed
Wide2800	GCC10/64	4/4	First = balanced direct
Wide2800	GCC15/1	0/4	Ordinary transposed
Wide2800	GCC15/64	4/4	First direct
Wide4600	GCC10/1	4/4	Ordinary transposed
Wide4600	GCC10/64	4/4	First = balanced direct
Wide4600	GCC15/1	0/4	Ordinary transposed
Wide4600	GCC15/64	4/4	Balanced direct

TABLE XI

MIN–MAX OVER EACH GROUP'S FOUR AMPLITUDE/REGIME CELLS, ROUNDED ONLY FOR DISPLAY. EACH PERCENTAGE IS $100(E_{left}/E_{named} - 1)$; NEGATIVE FAVORS THE LEFT. A REPEATED ROUNDED ENDPOINT CAN HIDE A NONZERO RANGE. DIRECT AND ORDINARY/HCUB PAIRS REMAIN KNOWN IN NATIVE/BLOCK64 DESPITE UNAVAILABLE CMSIS.

Design	Compiler/block	First / wide, %	Balanced / wide, %	Hcub / ordinary, %
Native	GCC10/1	−40.550 to −40.548	—	+0.448 to +0.448
Native	GCC10/64	−58.108 to −58.022	—	+0.791 to +0.794
Native	GCC15/1	−27.626 to −27.625	—	−1.454 to −1.454
Native	GCC15/64	−45.856 to −45.763	—	−1.987 to −1.983
Wide2800	GCC10/1	−34.339 to −34.321	−34.339 to −34.321	+7.656 to +7.657
Wide2800	GCC10/64	−54.969 to −54.882	−54.969 to −54.882	+7.220 to +7.222
Wide2800	GCC15/1	−24.616 to −24.586	−24.517 to −24.487	+0.117 to +0.117
Wide2800	GCC15/64	−43.107 to −43.020	−42.691 to −42.605	+0.129 to +0.129
Wide4600	GCC10/1	−29.294 to −29.277	−29.357 to −29.340	+4.512 to +4.513
Wide4600	GCC10/64	−45.743 to −45.653	−45.743 to −45.653	+3.368 to +3.374
Wide4600	GCC15/1	−23.741 to −23.727	−23.964 to −23.950	+0.571 to +0.572
Wide4600	GCC15/64	−42.254 to −42.167	−42.734 to −42.646	+1.377 to +1.382

and 31/22 post-startup; Wide4600/full reaches them 12/14 and 55/53 times, respectively. Counts apply only to the measured 257 or 1025 outputs, derive from exact source sums and do not identify executed target branch frequencies. Saturation coverage thus changes while the full-int16 arithmetic contract and the finite implementation decisions stay fixed.

All admitted executables respect the original memory limits. The maximum static RAM end is `0x20001210`, below `0x200017f0`; the largest conservative live stack chain is 688 bytes within the reserved 2048, with initial SP `0x20001ff0`. The maximum Flash end is `0x0800135e`. These linked-image and call-chain checks strengthen admission for the long fixtures; individual state, process-frame and selected-symbol metrics remain separate diagnostics.

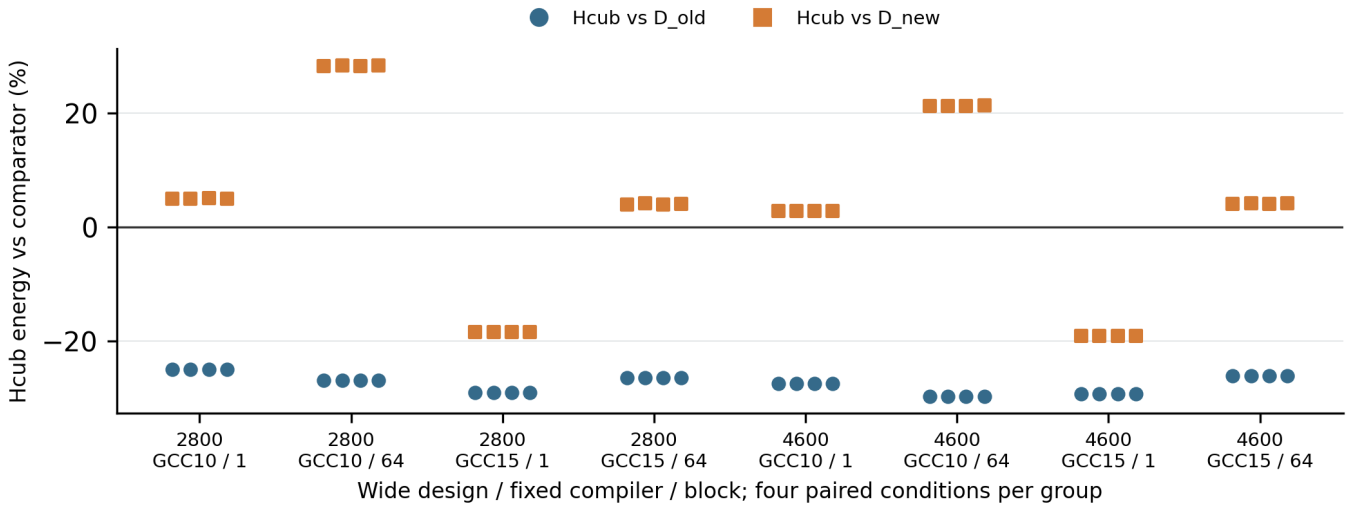


Fig. 3. Every complete wide-design fixed-compiler boundary. Hcub relative to the old and strengthened nontransposed minima crosses from below to above zero in 24 of 32 cells. GCC15/block1 supplies the eight nonreversals. Within each group, points run left to right as ADC/startup, ADC/post-startup, full/startup and full/post-startup. Eight unavailable native fast-CMSIS/block64 identities leave the full panel incomplete.

VIII. Discussion and limits

A. What the comparisons establish

The central lesson is that the baseline and the comparison being isolated can change the interpretation of a nominal winner. Correcting direct arithmetic reverses two original Hcub decisions and 24 conditions across the two fresh wide arrays. Yet Hcub loses to ordinary transposition throughout those fresh wide conditions. Its advantage over old direct/library controls therefore cannot establish a benefit from product sharing.

Compiler knowledge supplies a second mechanism. Making the one-output bound visible inside the direct callee removes identified preparation for another output, lowers GCC15 cost and reverses the compiler ranking. Other code changes accompany specialization, so the total saving cannot be assigned exclusively to sample rotation or to a particular compiler pass.

These findings support an empirical case study using established techniques. Exact event tradeoffs and portable evidence make its comparisons auditable; they do not add an optimization algorithm. Compiler-dependent winners, tied partitions and unavailable native cases also preclude a simple width-based selection rule.

B. Internal and external validity

The evaluated implementations form a finite candidate set. Arithmetic proofs, graph checks, UBSan, target output/state/guard checks, binary identity and emitted-code inspection address distinct failure modes; output agreement alone proves neither C safety nor model validity. The fixed cuts, schedules, graph seed and build options are controls, not an exhaustive optimization search. Freezing them before new costs prevents outcome-driven replacement.

The fresh panel has limited design breadth: three synthetic arrays at one length, including only two widening-required arrays. Repeated conditions do not establish application prevalence. The original ADC stream spans −1024 to 1023; $1024 \times 72509 = 74249216$ bounds output magnitude by 2266 without saturation. New full-range inputs exercise source clamps, but remain finite stimuli. Full-domain arithmetic legality does not make them exhaustive performance coverage, and the post-startup region is not proven steady state.

The results depend on the pinned simulator and cost configuration. Arm permits single-cycle or 32-cycle multipliers [11]; our cycle estimate assumes the single-cycle implementation. Zero waits, frequency and prefetch settings are also fixed. Board current, peripheral energy and physical uncertainty are unmeasured. Reweighting fixed events does not simulate another configuration, which could change execution itself. The block1 intervention likewise establishes no block64 behavior or ranking among other FIR forms.

Memory costs remain separate from modeled energy. Original direct state is 98/224 bytes at blocks 1/64, versus 192 bytes for transposition. Appendix A's process frames and selected symbols exclude complete call-chain and image costs. The fresh panel additionally checks linked-memory and conservative live-stack bounds. Summing these different diagnostics would misstate RAM use.

C. Evidence portability and reproduction levels

Saved-evidence portability is distinct from a fresh target rebuild. The saved 7,266-file package passed seven relocated verification commands covering numerical regeneration, historical controls, compiler explanation, event analysis, the panel and intervention. The checks had zero failures, integration skips or target-tool executions. Explicit copied roots prevented fallback to the original workspace. The package predates this editorial revision; its manifest and check report remain records of that snapshot.

Rebuilding requires separately acquired pinned GCC10/GCC15, simulator and CMSIS/Hcub dependencies; executable toolchains are excluded from the archive. Hashes establish byte consistency, not authenticity or physical validity. The package supports analysis regeneration and saved-binary verification, without claiming fresh performance replication, a public archive or a DOI. The separate corrected-timing report uses the same bound logs and binaries; it is a new saved-data analysis, outside that historical archive. All 304 admitted timing rows retain their energy totals. No panel pair changes its cycle ordering, and the twelve original event-analysis cycle differences and both intervention cycle reductions remain unchanged.

IX. Conclusion

Equalizing arithmetic-range information changes exact FIR implementation comparisons in this Cortex-M0 model case study. Certified native partials reverse two original Hcub decisions and 24 of 32 complete conditions across two fresh wide 48-tap arrays. Hcub nevertheless loses to ordinary transposition in every fresh wide condition, separating the baseline effect from incremental sharing. A matched block1 specialization also removes identified preparation for another output and reverses the direct compiler ranking.

The practical lesson is to examine arithmetic treatment, state representation and compiler-visible bounds together when interpreting an implementation comparison. The finite synthetic panel, unavailable native library cases and model-specific costs limit that lesson's reach. Broader coefficient/length coverage and physical validation of the key rankings are the next steps toward practical selection guidance.

Appendix A
Original principal-row resource diagnostics

Cycles are estimated per requested output. Code is selected kernel/coefficient symbols, excluding fixture and possibly helper code; frame is the compiler's static `probe_process` report; state is allocated state including block capacity. These are not total image, peak stack or SRAM high-water measurements.

TABLE XII
Every original principal-row resource diagnostic.

Cell	Form	Cycles/out.	Code, B	State, B	Frame, B
GCC10/1	CMSIS	1587.11	1494	112	8
GCC10/1	Wide direct	1607.11	1736	98	440
GCC10/1	Partial direct	1066.11	1036	98	72
GCC10/1	Ordinary transposed	1107.11	1540	192	272
GCC10/1	Hcub	1151.11	1504	192	344
GCC10/64	CMSIS	924.36	1494	236	8
GCC10/64	Wide direct	1029.35	1720	224	440
GCC10/64	Partial direct	583.33	1196	224	72
GCC10/64	Ordinary transposed	630.66	1540	192	272
GCC10/64	Hcub	636.42	1504	192	344
GCC15/1	CMSIS	1552.11	1450	112	8
GCC15/1	Wide direct	1799.11	1872	98	608
GCC15/1	Partial direct	1366.64	1372	98	240
GCC15/1	Ordinary transposed	1101.11	1532	192	272
GCC15/1	Hcub	1063.11	1352	192	344
GCC15/64	CMSIS	852.02	1450	236	8
GCC15/64	Wide direct	1074.67	1884	224	608
GCC15/64	Partial direct	612.42	1312	224	232
GCC15/64	Ordinary transposed	627.59	1532	192	272
GCC15/64	Hcub	593.51	1352	192	344

Appendix B
Editable evidence map

The first manuscript and its results remain under `research/fir-paper/`; its source manifest records the six frozen numerical inputs. The saved-event analysis contains all 12 difference vectors, cost/resource components, exact rational boundaries and endpoint winners. The artifact instructions, manifest and relocated check report describe the saved package and verification levels. The corrected timing report supersedes historical cycle fields for this manuscript's 20 original, 280 admitted panel and four intervention performance identities. Its reanalysis instructions identify the formula, raw-log bindings and instruction-subset checks. Eight unavailable panel identities remain unavailable.

The frozen protocol, design, inputs, certificates, graphs and sources bind the fresh panel. Its inventory, summary and raw audit retain every condition, event vector, tie and unavailable case. The intervention inventory binds its eight rows to raw artifacts. A separate manual code inspection records the preparation-removal assessment; the automated record's pending manual-review field does not supersede it. These internal checks and agent reviews do not constitute external peer review. The historical 12/24 filter-bank statistic belongs to its original candidate set; no revised bank-wide result is inferred here.

References

[1] L. De Coster, M. Adé, R. Lauwereins, and J. A. Peperstraete, "Code Generation for Compiled Bit-True Simulation of DSP Applications," in *Proceedings of the 11th International Symposium on System Synthesis*, 1998, pp. 9–14. Primary manuscript.

[2] A. Gačić, M. Püschel, and J. M. F. Moura, "Fast Automatic Software Implementations of FIR Filters," in *Proceedings of IEEE ICASSP*, vol. II, 2003, pp. 541–544, doi: 10.1109/ICASSP.2003.1202423. Author-uploaded full text. The author-group manuscript provides an additional primary-source location.

[3] P. D'Alberto, M. Püschel, and F. Franchetti, "Performance/Energy Optimization of DSP Transforms on the XScale Processor," in *HiPEAC 2007*, LNCS 4367, 2007, pp. 201–214. Author manuscript.

[4] Y. Voronenko and M. Püschel, "Multiplierless Multiple Constant Multiplication," *ACM Transactions on Algorithms*, vol. 3, no. 2, Art. 11, May 2007, doi: 10.1145/1240233.1240234. Author manuscript.

[5] K. Nikov, K. Georgiou, Z. Chamski, K. Eder, and J. Nunez-Yanez, "Accurate Energy Modelling on the Cortex-M0 Processor for Profiling and Static Analysis," in *2022 29th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, 2022, pp. 1–4, doi: 10.1109/ICECS202256217.2022.9971086. Accessible version: arXiv:2301.12806v1, 2023. Primary paper.

[6] D. Ménard and O. Sentieys, "DSP Code Generation with Optimized Data Word-Length Selection," in *SCOPES 2004*, 2004, pp. 214–228, doi: 10.1007/978-3-540-30113-4_16. Author-uploaded manuscript.

[7] GNU Project, "Optimize Options," *GCC Online Documentation*. Official documentation.

[8] SciPy developers, "scipy.signal.minimum_phase," *SciPy 1.15.3 Reference*. Pinned API documentation.

[9] Arm, "Finite Impulse Response (FIR) Filters," *CMSIS-DSP Documentation*. Official arithmetic/storage guidance. Current guidance supplies context; the artifact's pinned v1.14.4 source and compiler admission govern the actual library paths.

[10] ACM SIGSIM-PADS 2026, "Artifact Evaluation." Official artifact guidance. Used as an established practice reference, not a binding venue rule.

[11] Arm, *Cortex-M0 Technical Reference Manual*, DDI 0432C, §§1.4.1 and 3.3, 2009. Configurable multiplier and instruction timings.

[12] GNU Project, "Integers," *GCC 10.3 and 15.2 Online Documentation*, §4.5. GCC 10.3 integer semantics; GCC 15.2 integer semantics.